

Tradução e Geração de Código

Visão Geral

A fase de tradução converte a AST verificada semanticamente em código assembly para a EWVM (Educational Web Virtual Machine). É composta por quatro módulos:

```
AST verificada
  ↓
[IRBuilder]    → IRProgram (representação intermédia)
  ↓
[IROptimizer]  → IRProgram otimizado
  ↓
[IRCodeGen]    → string com código EWVM
  ↓
ficheiro .ewvm
```

O **Translator** orquestra os três passos, expondo uma interface única `translate(ast) -> str`.

Representação Intermédia (IR)

Motivação e vantagens

A introdução de uma representação intermédia entre a AST e o código final traz várias vantagens:

1. **Separação de responsabilidades:** a geração de código (IRBuilder) não precisa de conhecer o formato textual da EWVM; o gerador de texto (IRCodeGen) não precisa de conhecer a AST.
2. **Optimizações independentes:** o otimizador opera sobre uma sequência plana de instruções, mais simples do que trabalhar sobre a AST ou sobre texto.
3. **Testabilidade:** a IR pode ser inspeccionada e testada independentemente do output final.
4. **Portabilidade potencial:** substituir o IRCodeGen por outro gerador permitiria emitir código para uma máquina diferente sem tocar no IRBuilder.

Estrutura

```
IRProgram
├── IRUnit[]          (uma por PROGRAM / FUNCTION / SUBROUTINE)
│   └── IRInstr[]     (sequência linear de instruções)
```

```
@dataclass
class IRUnit:
    name: str
    kind: str          # 'PROGRAM' | 'FUNCTION' | 'SUBROUTINE'
    instrs: list[IRInstr]
```

```
@dataclass
class IRProgram:
    units: list[IRUnit]
```

Categorias de instruções IR

Literais (push de constantes)

Instrução IR	Mnemónico EWVM	Descrição
IRPushI(val)	PUSHI n	Empilha inteiro
IRPushF(val)	PUSHF x	Empilha real
IRPushS(val)	PUSHS "s"	Empilha string (na string heap)
IRPushN(n)	PUSHN n	Empilha n zeros (alocação de frame local/global)

Acesso a variáveis

Instrução IR	Mnemónico EWVM	Descrição
IRPushG(idx)	PUSHG n	Lê da posição gp[n] (variável global)
IRPushL(idx)	PUSHL n	Lê da posição fp[n] (variável local; n pode ser negativo para parâmetros)
IRStoreG(idx)	STOREG n	Escreve em gp[n]
IRStoreL(idx)	STOREL n	Escreve em fp[n]

Heap (arrays dinâmicos)

Instrução IR	Mnemónico EWVM	Descrição
IRAlloc(n)	ALLOC n	Aloca bloco de n slots na heap; empilha endereço
IRLoadN()	LOADN	Pilha [addr, idx] → addr[idx]
IRStoreN()	STOREN	Pilha [addr, idx, val] → addr[idx] = val

Manipulação de pilha

Instrução IR	Mnemónico EWVM	Descrição
IRPop(n)	POP n	Descarta n valores do topo
IRSwap()	SWAP	Troca os dois valores no topo
IRDup(n)	DUP n	Duplica o topo n vezes

Instrução IR	Mnemónico EWVM	Descrição
IRCopyK(n)	COPY n	Copia os n valores do topo e empilha-os na mesma ordem

Aritmética inteira e real

Inteira	Real	Operação
IRAdd / ADD	IRFAdd / FADD	Adição
IRSub / SUB	IRFSub / FSUB	Subtração
IRMul / MUL	IRFMul / FMUL	Multiplicação
IRDiv / DIV	IRFDiv / FDIV	Divisão
IRMod / MOD	—	Resto (só inteiros)

Comparações

Inteira	Real	Operação
IREqual / EQUAL	—	Igualdade (partilhada)
IRInf / INF	IRFInf / FINF	Menor que
IRInfEq / INFEQ	IRFInfEq / FINFEQ	Menor ou igual
IRSup / SUP	IRFSup / FSUP	Maior que
IRSupEq / SUPEQ	IRFSupEq / FSUPEQ	Maior ou igual

A operação **.NE** (diferente de) não tem instrução nativa na EWVM — é emitida como **EQUAL** seguido de **NOT**.

Lógica

IR	EWVM	Descrição
IRAnd	AND	E lógico
IROr	OR	Ou lógico
IRNot	NOT	Negação (0 → 1, outro → 0)

Conversões de tipo

IR	EWVM	Descrição
IRItOf	ITOf	Inteiro → real
IRFtoI	FTOI	Real → inteiro (trunca)
IRAtoi	ATOI	String heap → inteiro

IR	EWVM	Descrição
<code>IRAtof</code>	<code>ATOF</code>	String heap → real

Controlo de fluxo

IR	EWVM	Descrição
<code>IRLabel(name)</code>	<code>name:</code>	Definição de label
<code>IRJump(label)</code>	<code>JUMP label</code>	Salto incondicional
<code>IRJZ(label)</code>	<code>JZ label</code>	Salta se topo == 0
<code>IRPushA(label)</code>	<code>PUSHA label</code>	Empilha endereço do label
<code>IRCall()</code>	<code>CALL</code>	Chama subprograma (endereço no topo)
<code>IRReturn()</code>	<code>RETURN</code>	Retorna ao chamador

I/O

IR	EWVM	Descrição
<code>IRRead</code>	<code>READ</code>	Lê string do stdin
<code>IRWriteI</code>	<code>WRITEI</code>	Imprime inteiro
<code>IRWriteF</code>	<code>WRITEF</code>	Imprime real
<code>IRWriteS</code>	<code>WRITES</code>	Imprime string
<code>IRWriteLn</code>	<code>WRITELN</code>	Imprime newline

Controlo de programa

IR	EWVM	Descrição
<code>IRStart</code>	<code>START</code>	Inicializa <code>fp = sp</code> (início do programa principal)
<code>IRStop</code>	<code>STOP</code>	Termina o programa

Geração de IR (`IRBuilder`)

O `IRBuilder` percorre a AST com o padrão Visitor e emite instruções IR para a `IRUnit` activa.

Layout do stack frame

Programa principal (scope global)

```
gp[0]  primeira variável declarada
gp[1]  segunda variável
```

```
...
gp[N-1] N-ésima variável
```

Variáveis globais são acedidas com **PUSHG/STOREG**.

Funções e subrotinas (scope local)

```
...stack do chamador...
[slot de retorno] ← empilhado pelo chamador antes dos args (só funções)
fp[-arity] último argumento
...
fp[-2] segundo argumento
fp[-1] primeiro argumento (o mais próximo do topo antes do CALL)
— fp aponta aqui (frame pointer) —
fp[0] var. de retorno (só funções) / primeiro local
fp[1] segundo local
...
fp[N-1] N-ésimo local
```

Esta convenção permite que o código gerado aceda a parâmetros com índices negativos e variáveis locais com índices não-negativos, usando sempre **PUSHL/STOREL**.

Padrão de chamada de funções

Antes de um **CALL**:

1. **PUSHI 0** — reserva o slot de retorno (será preenchido pela função).
2. Argumentos em **ordem inversa** (o último arg é empilhado primeiro → fica em **fp[-arity]**; o primeiro arg fica em **fp[-1]**).
3. **PUSHA nome_funcao + CALL**.

Após o **RETURN** da função:

4. **POP arity** — remove os argumentos.
5. O slot de retorno (que a função escreveu com **STOREL -(arity+1)**) fica no topo.

Para subrotinas, não há slot de retorno (passo 1 omitido); após o **RETURN**, o chamador faz **POP arity**.

Prologue de funções/subrotinas

```
nome_funcao:
  PUSHN n_locals      (se n_locals > 0)
  ALLOC dim; STOREL idx (para cada array local)
  ...body...
```

O **PUSHN n_locals** aloca todos os slots locais de uma vez. As alocações de heap para arrays seguem imediatamente.

Epilogue de funções

```
def _emit_function_return(self) -> None:
    ret_sym = self._symbols.lookup_var(self._current_unit)
    # Copia variável de retorno para o slot reservado pelo chamador
    self._emit(IRPushL(ret_sym.index))           # fp[0]
    self._emit(IRStoreL(-(self._current_arity + 1))) # → slot de
    retorno
    # Limpa os locais
    if n_locals > 0:
        self._emit(IRPop(n_locals))
```

Código gerado para cada construção

Atribuição a variável escalar

```
<eval RHS>
[ITOF | FTOI se conversão necessária]
STOREG/STOREL idx
```

Atribuição a elemento de array

```
PUSHG/PUSHL addr_idx    ← endereço da heap
<eval índice>
PUSHI 1
SUB                      ← converte 1-based → 0-based
<eval RHS>
[ITOF | FTOI se conversão necessária]
STOREN
```

IF-THEN-[ELSE]-ENDIF

```
<eval condição>
JZ ELSElabelN
<then_body>
JUMP ENDIFlabelN
ELSElabelN:
    <else_body>           ← (vazio se sem ELSE)
ENDIFlabelN:
```

DO loop

```

    <eval start>
    STOREG/STOREL var
D0labelN:
    PUSHG/PUSHL var
    <eval stop>
    INFEQ                ← var <= stop
    JZ ENDD0labelN
    <body>
    PUSHG/PUSHL var
    <eval step | PUSHI 1>
    ADD
    STOREG/STOREL var
    JUMP D0labelN
ENDD0labelN:

```

Nota: o DO loop usa sempre **INFEQ** (inteiro). Se a variável de controlo for **REAL**, a verificação deveria usar **FINFEQ** — esta é uma limitação identificada.

GOTO

```
JUMP labelN
```

O label gerado para **GOTO N** é **labelN**, consistente com o prefixo gerado por **visit_LabeledStmt**.

Operador ** (potência)

A EWVM não tem instrução de potência. A solução foi implementar uma função auxiliar **POWERFUNClabel** gerada sob pedido:

```

POWERFUNClabel:
    PUSHN 2          (result=fp[0], i=fp[1])
    PUSHI 1; STOREL 0  result = 1
    PUSHI 1; STOREL 1  i = 1
POWERlabel:
    PUSHL 1          i
    PUSHL -2         exp (fp[-2])
    INFEQ           i <= exp?
    JZ POWERendlabel
    PUSHL 0          result
    PUSHL -1         base (fp[-1])
    MUL
    STOREL 0         result = result * base
    PUSHL 1; PUSHI 1; ADD; STOREL 1  i++
    JUMP POWERlabel
POWERendlabel:
    PUSHL 0
    STOREL -3        escrita no slot de retorno (arity=2 → fp[-(2+1)])

```

```
POP 2
RETURN
```

O flag `_needs_power` é activado quando o `IRBuilder` encontra um nó `ArithmeticBinOp` com `op == '**'`. O `IRCodeGen` emite a função auxiliar no final do output apenas se este flag estiver activo.

Funções intrínsecas MAX/MIN

Implementadas com comparação em pilha usando `COPY 2`:

```
<eval arg1>
<eval arg2>
COPY 2          [a, b, a, b]
SUP/INF         [a, b, (a>b)]
JZ MINMAXlabelN se falso, b é o máximo
POP 1           fica a
JUMP MINMAXENDlabelN
MINMAXlabelN:
  SWAP; POP 1    fica b
MINMAXENDlabelN:
  ...           (repete para arg3, arg4, ...)
```

Para mais de 2 argumentos, o padrão repete-se iterativamente: o resultado parcial fica no topo e é comparado com o próximo argumento.

SQRT

A EWVM não suporta raiz quadrada. A implementação actual emite:

```
<eval argumento>
ITOF (se necessário)
PUSHS "SQRT not supported by EWVM"
WRITES
STOP
```

Esta é uma limitação declarada — o programa aborta com mensagem se `SQRT` for invocado.

Optimizações (`IROptimizer`)

O optimizador opera sobre a IR por **passagens de peephole** com **iteração até ponto fixo**:

```
for _ in range(20):
    new, changed = self._peephole(instrs)
    new = self._dead_code(new)
    new = self._trivial_jumps(new)
```



```
new = self._unused_labels(new, protected)
if not changed and new == instrs:
    break
```

O limite de 20 iterações é uma salvaguarda contra ciclos infinitos. Na prática, a convergência é rápida (1–3 iterações).

Passagem 1: Constant Folding (peephole)

Detecta sequências **PUSHI/PUSHF**, **PUSHI/PUSHF**, **operação binária** e substitui pela constante resultante:

```
PUSHI 3      →   PUSHI 7
PUSHI 4
ADD
```

Cobre operações inteiras (**ADD**, **SUB**, **MUL**, **DIV**, **MOD**, comparações, **AND**, **OR**) e reais (**FADD**, **FSUB**, **FMUL**, **FDIV**, comparações float). A promoção inteiro → real é feita automaticamente quando um dos operandos é **PUSHF**.

Também dobra **PUSHI n**; **NOT** → **PUSHI (n==0 ? 1 : 0)**.

Divisão por zero é protegida (**return None** → não dobra).

Passagem 2: Eliminação de código morto

Remove instruções inacessíveis após saltos incondicionais, **RETURN** ou **STOP**, até ao próximo label:

```
JUMP fim      →   JUMP fim
PUSHI 42      (removido)
fim:          fim:
```

Passagem 3: Eliminação de saltos triviais

Remove um **JUMP** imediatamente seguido do label de destino:

```
JUMP next     →   next:
next:
```

Esta optimização é iterativa: pode ser necessária após a eliminação de código morto expor novos saltos triviais.

Passagem 4: Eliminação de labels órfãos

Remove labels que não são referenciados por nenhum **JUMP**, **JZ**, ou **PUSHA** na unidade ou no programa global. Labels de subprogramas (referenciados por **PUSHA** noutras unidades) são protegidos pelo conjunto

global_pusha:

```
global_pusha: set[str] = {
    instr.label
    for unit in program.units
    for instr in unit.instrs
    if isinstance(instr, IRPushA)
}
```

Geração de Texto (IRCodeGen)

Converte cada **IRInstr** para o mnemónico EWVM correspondente. A conversão é uma cadeia de **isinstance** que mapeia directamente cada classe IR para uma string. Strings **PUSHS** têm as aspas duplas internas escapadas.

O formato real dos números reais usa o especificador **:g** do Python, que produz a representação mais compacta (sem zeros desnecessários).

Translator — Orquestrador

```
class Translator:
    def translate(self, ast: Program) -> str:
        ir, needs_power = self._builder.build(ast)
        ir = self._optimizer.optimize(ir)
        return self._codegen.generate(ir, needs_power)
```

A interface é intencionalmente simples: recebe uma AST, devolve código EWVM como string.

Limitações e Notas de Implementação

- **Potência inteira apenas:** **POWERFUNClabel** usa **MUL** inteiro, pelo que ****** com base ou expoente real não é suportado correctamente.
- **DO loop com REAL:** a comparação no DO usa **INFEQ** (inteiro) mesmo quando a variável de controlo é **REAL**; deveria usar **FINFEQ**.
- **Passagem de parâmetros por valor:** os argumentos são copiados para a stack; modificações dentro de subprogramas não afectam as variáveis do chamador (ao contrário do Fortran 77 padrão, que usa passagem por referência).
- **Sem suporte a SQRT em runtime:** a instrução gera código que termina o programa com mensagem de erro.
- **Arrays unidimensionais:** apenas arrays com uma dimensão são suportados em toda a pipeline.